

MODULE 02

Local BLAST+ & Reference Databases

eDNA Bioinformatics Learning Series · Yaw Baah

Module 01 explained *what* BLAST is and how it scores alignments. This module answers the next practical question: **how do you run BLAST on your own machine**, against a database you control, without rate limits, internet delays, or reproducibility surprises? By the end you will understand local databases, the BLAST+ toolchain, how the pipeline's **--local** flag works, and why database version stamping matters for scientific credibility.

What you will learn

1. Web vs Local BLAST — when to use each
2. How BLAST+ database files are structured
3. Which database to choose for eDNA
4. Setting up a database with scripts/setup_db.py
5. Running the pipeline in --local mode
6. Threading and performance
7. Database version stamping for reproducibility
8. Glossary

1. Web BLAST vs Local BLAST+ — Choose the Right Mode

The pipeline ships with two BLAST backends. Both produce identical XML output — the same parser reads both. The choice of backend affects **speed**, **reproducibility**, and **cost**, not the biology.

Feature	Web BLAST (NCBI)	Local BLAST+
Speed	30 – 120 s per sequence (queue-dependent)	Seconds on SSD; milliseconds for 16S
Rate limit	3 requests/sec max (blocked if exceeded)	None — limited only by your CPU
Internet required	Yes	No
Database version	Updated daily — changes between runs	Fixed to your local copy — fully reproducible
Database size	Always up to date, no local disk needed	nt ~300 GB; 16S ~1 GB; SILVA SSU ~7 GB
Best for	Quick checks, small datasets, no BLAST+ installed	Production runs, large datasets, offline environments
Pipeline flag	--email you@example.com	--local --db-path /data/blast/nt

Rule of thumb

Use **web BLAST** when you are testing a handful of sequences and do not need to reproduce the exact same database version. Switch to **local BLAST+** for any run that will go into a report, paper, or client deliverable — the version-locked database guarantees the same result if you re-run six months later.

2. How BLAST+ Database Files Are Structured

A BLAST database is not a single file — it is a family of binary files that together form an indexed, searchable copy of millions of sequences. Understanding the structure helps you diagnose errors (wrong path, corrupted download, wrong database type).

Nucleotide database files (blastn uses these):

Extension	What it contains
.nsq	Sequence data — the raw nucleotide letters, compressed
.nin	Index — maps sequence IDs to positions in .nsq
.nhl	Header — human-readable titles for each sequence (BLAST hit titles)
.nsi / .nsd	String index — for text searches by accession or title
.nal	Alias file — for multi-volume databases (nt is split across volumes)
.njs (BLAST5)	JSON metadata — database title, date, BLAST version used to build

When you pass `--db-path /data/blast/nt` to the pipeline, BLAST+ automatically finds all the volume files (nt.00.nsq, nt.01.nsq ...) via the `nt.nal` alias file. You never need to specify individual volumes.

Checking a database with `blastdbcmd`

`blastdbcmd` is a diagnostic tool bundled with BLAST+.

```
blastdbcmd -db /data/blast/nt -info
```

This prints the database title, creation date, number of sequences, and total base count. The pipeline calls this automatically and saves the result to `data/results/db_version.json` every time you run with `--local`.

3. Choosing the Right Reference Database

The database you choose has a bigger impact on your results than any parameter. A sequence that perfectly matches an organism in SILVA but has no representative in nt will return *unclassified* if you use nt. Below are the four databases most relevant to freshwater / environmental eDNA.

Database	Best for	Size	Notes
NCBI nt	General purpose — any organism	~300 GB	Broadest coverage; includes all GenBank sequences; very large
16S_ribosomal_RNA	Bacteria and Archaea (16S marker)	~1 GB	Curated NCBI set; much faster than nt; install in minutes
ITS_RefSeq_Fungi	Fungi (ITS1/ITS2 marker)	~300 MB	Used for mycology eDNA; small and fast
SILVA 138 SSU	Eukaryotes, bacteria, archaea (18S/16S)	~7 GB	Gold standard for rRNA; must be downloaded from arb-silva.de

Decision guide for UK pond eDNA:

- **Phytoplankton / microalgae (18S marker)** → SILVA 138 SSU or NCBI nt
- **Bacteria / cyanobacteria (16S marker)** → 16S_ribosomal_RNA (fastest) or SILVA 138
- **Fungi (ITS marker)** → ITS_RefSeq_Fungi
- **Macroinvertebrates / fish / plants (COI or rbcL)** → NCBI nt (only comprehensive option)
- **Unknown / mixed marker** → NCBI nt (safest fallback)

4. Setting Up a Local Database — `scripts/setup_db.py`

The pipeline includes `scripts/setup_db.py` — a one-command wrapper around BLAST+'s built-in `update_blastdb.pl` utility. You do not need to manually navigate NCBI's FTP servers.

1. Install BLAST+	Ubuntu/Debian: <code>sudo apt-get install ncbi-blast+</code> macOS: <code>brew install blast</code> Codespaces: already installed in the devcontainer. Verify with: <code>blastn -version</code>
2. Download a database	Run: <code>python scripts/setup_db.py --db 16S_ribosomal_RNA --dest /data/blast/</code> for a quick (~1 GB) start. For the full nt database (300 GB): <code>python scripts/setup_db.py --db nt --dest /data/blast/</code> (allow 1–6 hours on a fast connection).
3. Check what is installed	Run: <code>python scripts/setup_db.py --list --dest /data/blast/</code> to see all databases already present and their titles.
4. Run the pipeline	Run: <code>python pipeline.py --fasta data/sample_sequences.fasta --local --db-path /data/blast/16S_ribosomal_RNA --threads 4 No --email</code> needed in local mode.

SILVA 138 — manual download

SILVA is not available via `update_blastdb.pl`. Download the FASTA from arb-silva.de/download/arb-files/ (file: `SILVA_138_SSURef_NR99_tax_silva.fasta.gz`), then format it with:

```
makeblastdb -in SILVA_138_SSURef_NR99_tax_silva.fasta -dbtype nucl -title
SILVA_138 -out /data/silva/SILVA_138_SSU
```

Then use: **`--db-path /data/silva/SILVA_138_SSU`**

5. How the Pipeline's --local Mode Works

When you pass **--local**, `pipeline.py` calls **src/local_blast.py** instead of `src/blast_query.py`. Everything downstream — XML parsing, species extraction, LCA resolution, diversity metrics — is identical. The switch is invisible to the biology.

What `src/local_blast.py` does:

Step	Detail
Binary check	Locates the <code>blastn</code> binary using <code>shutil.which()</code> . Prints clear installation instructions if not found.
DB version stamp	Calls <code>blastdbcmd -info</code> and writes <code>db_version.json</code> to your results folder. Recorded once before any queries.
Cache check	Skips a query if its <code>.xml</code> file already exists and is non-empty — identical to web mode.
BLAST via subprocess	Passes the FASTA sequence on <code>stdin</code> (no temp files). Writes XML directly to the <code>results/xml/</code> folder.
Timeout guard	Kills a stalled BLAST process after 300 seconds with a clear error message.
Threading	Runs <code>N</code> workers in parallel using <code>ThreadPoolExecutor</code> when <code>--threads N > 1</code> . Each query writes its own file — no race conditions.

Caching explained:

Caching is critical for large datasets. If your run is interrupted at sequence 847 of 1000, re-running the pipeline picks up at 848 — the 847 completed XML files are reused instantly. This is the same behaviour as the web BLAST backend — the pipeline never wastes a query it has already computed.

Command reference

```
# Minimal local run:
python pipeline.py --fasta data/sample_sequences.fasta --local --db-path
/data/blast/16S_ribosomal_RNA

# With threading and strict e-value:
python pipeline.py --fasta data/sample_sequences.fasta --local --db-path
/data/blast/nt --threads 8 --evalue 1e-20 --max-hits 10

# Keep only best hit per sequence:
python pipeline.py --fasta data/sample_sequences.fasta --local --db-path
/data/blast/nt --threads 4 --top-hit-only
```

6. Threading and Performance

Local BLAST+ supports parallel execution — you can run multiple sequences simultaneously across CPU cores. The pipeline exposes this via the **--threads N** flag.

Choosing the right thread count:

Storage type	Recommended --threads
Spinning HDD	1 (sequential only — parallel seeks thrash the drive)
SATA SSD	2 – 4 (I/O is fast enough; don't exceed CPU core count)
NVMe SSD	4 – 8 (high bandwidth; safe to use all cores)
GitHub Codespaces / devcontainer	4 – 8 (NVMe-backed; BLAST+ pre-installed)

Each thread runs a completely independent BLAST process — one per query sequence. They write to separate XML files so there are no shared-state issues. If any worker fails (e.g. timeout), the pipeline reports all failures together rather than silently dropping sequences.

7. Database Version Stamping — Why It Matters

NCBI updates the nt database **daily**. A sequence that returns *Chlorella vulgaris* today might return *Chlorella sorokiniana* in three months if NCBI's curators reclassify a reference entry. For regulatory eDNA surveys (e.g. protected species detection), this non-reproducibility is a serious problem.

The solution is simple: **lock the database version**. With local BLAST+ you download a database snapshot and never update it mid-project. The pipeline automatically records which snapshot you used:

data/results/db_version.json — written on every local run

```
{ "db_path": "/data/blast/nt", "db_version": "Database: nt | Date: Feb 14, 2025", "program": "blastn", "stamped_at": "2026-05-22T09:00:00Z" }
```

This file travels with your results CSV. Anyone auditing your data — a regulator, a journal reviewer, or yourself in a year's time — can see exactly which database produced which identifications.

Best practice: archive the db_version.json file alongside your blast_hit_table.csv. If you ever re-run the analysis, compare the new version stamp against the archived one — any difference means results could legitimately differ.

8. Glossary

BLAST+

The NCBI command-line suite of BLAST programs: `blastn`, `blastx`, `tblastn`, `blastdbcmd`, `makeblastdb`, `update_blastdb.pl`. The '+' distinguishes the modern C++ rewrite from the original legacy BLAST.

blastn

Nucleotide vs nucleotide BLAST. The main program used by this pipeline. Takes a DNA query and searches a nucleotide database.

blastdbcmd

A utility for querying and inspecting local BLAST databases. Used by the pipeline to extract the database version stamp.

makeblastdb

Converts a plain FASTA file into the binary BLAST database format (`.nsq` / `.nin` / `.nhr` files). Used when building a custom database (e.g. SILVA).

update_blastdb.pl

A Perl script bundled with BLAST+ that downloads and decompresses pre-built NCBI databases. Wrapped by `scripts/setup_db.py`.

.nal file

Database alias file. Points to all volumes of a multi-part database (e.g. `nt.00` through `nt.99`). BLAST+ reads the alias and automatically searches all volumes.

BLAST5 format

The current binary database format introduced in BLAST+ 2.10. Each database includes a `.njs` metadata file in JSON format.

--outfmt 5

The BLAST command-line flag that requests XML output. The pipeline always uses this format because Biopython's parser reads it reliably without ambiguity.

Version stamping

Recording the exact database title and creation date alongside results so the analysis can be reproduced or audited in the future.

ThreadPoolExecutor

Python's `concurrent.futures` class used to run multiple BLAST processes in parallel. Each worker is an independent process — no shared memory.

Cache hit

When the pipeline finds an existing non-empty `.xml` file for a query and skips the BLAST search entirely. Prevents redundant computation on re-runs.

db_version.json

A JSON file written to `data/results/` on every local BLAST run. Records `db_path`, `db_version` string, program, and UTC timestamp.

Key Takeaways

- **Local BLAST+ is the production choice.** Faster, no rate limits, fully offline — use it for any run that matters.
- **Database choice matters more than parameters.** Pick the database that covers your marker gene and target organisms.
- **Small databases first.** 16S (~1 GB) or ITS (~300 MB) let you test everything before committing to nt (~300 GB).
- **Version-stamp every run.** `db_version.json` is your audit trail — commit it alongside your results.
- **Threading scales with storage speed.** `--threads 4` is a safe default on SSDs; `--threads 1` is safest on spinning hard drives.

Next: Module 03 — Regulatory PDF Reports & Data Visualisation